# API SECURITY ASSESSMENT REPORT

OWASP crAPI — Completely Ridiculous API

| | |
|---|---|
| **Target System** | OWASP crAPI (Completely Ridiculous API) |
| **Deployment Environment** | Docker (localhost:8888) |
| **Author** | David Msekena |
| **Internship** | Future Interns |
| **Institution** | Institute of Finance Management (IFM), Dar es Salaam |
| **Programme** | BSc Cybersecurity |
| **Assessment Type** | API Security Assessment / Penetration Testing |
| **Date** | April 2026 |
| **Classification** | Confidential Academic Use Only |

## TOOLS USED IN THE ASSESSMENT

| TOOL | USAGE |
|---|---|
| Postman | Testing the vulnerabilities |
| Browser Dev Tools (network, console) | Register/Login and watching Network tab to see correct requests & OTPs |
| Docker | Running crAPI |

## Contents

TOOLS USED IN THE ASSESSMENT ..... 1
1. Executive Summary ..... 3
Vulnerability Summary ..... 3
2. Introduction ..... 4
2.1 Application Overview ..... 4
3. Scope ..... 6
4. Methodology ..... 7
5. Vulnerabilities Found ..... 8
5.1. API Endpoint Discovery ..... 8
5.4. Excessive Data Exposure / Information Disclosure ..... 15
5.5. Weak Tokens (JWT & OTP Brute Force) ..... 17
5.6. Missing Rate Limiting ..... 19
5.7. Mass Assignment ..... 21
6. Conclusion ..... 24

## 1. Executive Summary

This report presents the findings of an API security assessment conducted on the OWASP crAPI (Completely Ridiculous API) platform, a deliberately vulnerable API designed for security training purposes. The assessment was performed in a controlled Docker environment hosted on localhost:8888, using industry-standard tools including Postman and browser developer tools.

A total of seven (7) vulnerabilities were identified spanning a range of severity levels. These include critical issues such as Broken Object Level Authorization (BOLA), Mass Assignment, and Weak Token handling, as well as high-severity concerns including Missing Rate Limiting, Exposed Information Disclosure, and API Endpoint Discovery. Each vulnerability was manually tested and documented with supporting evidence screenshots.

The findings align with well-known OWASP API Security Top 10 categories, demonstrating real-world attack vectors that could lead to unauthorized data access, privilege escalation, and system abuse. This report provides detailed risk classifications, technical evidence, and actionable remediation recommendations for each identified vulnerability.

### Vulnerability Summary

| # | Vulnerability | OWASP Category | Severity | Status |
|---|---|---|---|---|
| 1 | API Endpoint Discovery | API9:2023 | **MEDIUM** | Confirmed |
| 2 | Broken Object Level Authorization (BOLA) | API1:2023 | **HIGH** | Confirmed |
| 3 | Broken Authorization (Data Access) | API1:2023 | **HIGH** | Confirmed |
| 4 | Excessive Data Exposure / Info Disclosure | API3:2023 | **HIGH** | Confirmed |
| 5 | Weak Tokens — JWT & OTP Brute Force | API2:2023 | **CRITICAL** | Confirmed |
| 6 | Missing Rate Limiting | API4:2023 | **HIGH** | Confirmed |
| 7 | Mass Assignment / Input Validation Failure | API6:2023 | **CRITICAL** | Confirmed |

## 2. Introduction

This assessment was conducted on the OWASP crAPI (Completely Ridiculous API) an intentionally vulnerable API application developed by OWASP specifically for security awareness and training. The platform simulates a vehicle management web application and exposes a range of real-world API vulnerabilities commonly found in production systems. The crAPI application was deployed locally using Docker and accessed at http://localhost:8888.

The primary objective of this assessment is to identify, analyze, and document security vulnerabilities present in the crAPI API surface. Each finding is evaluated against the OWASP API Security Top 10 (2023) framework, classified by severity using a standardized risk scale, and accompanied by practical remediation guidance.

### 2.1 Application Overview

The following screenshots show the crAPI web application interface as observed during the assessment. The application includes a sign-in page and a sign-up page, both of which interact with the underlying API endpoints that were tested.

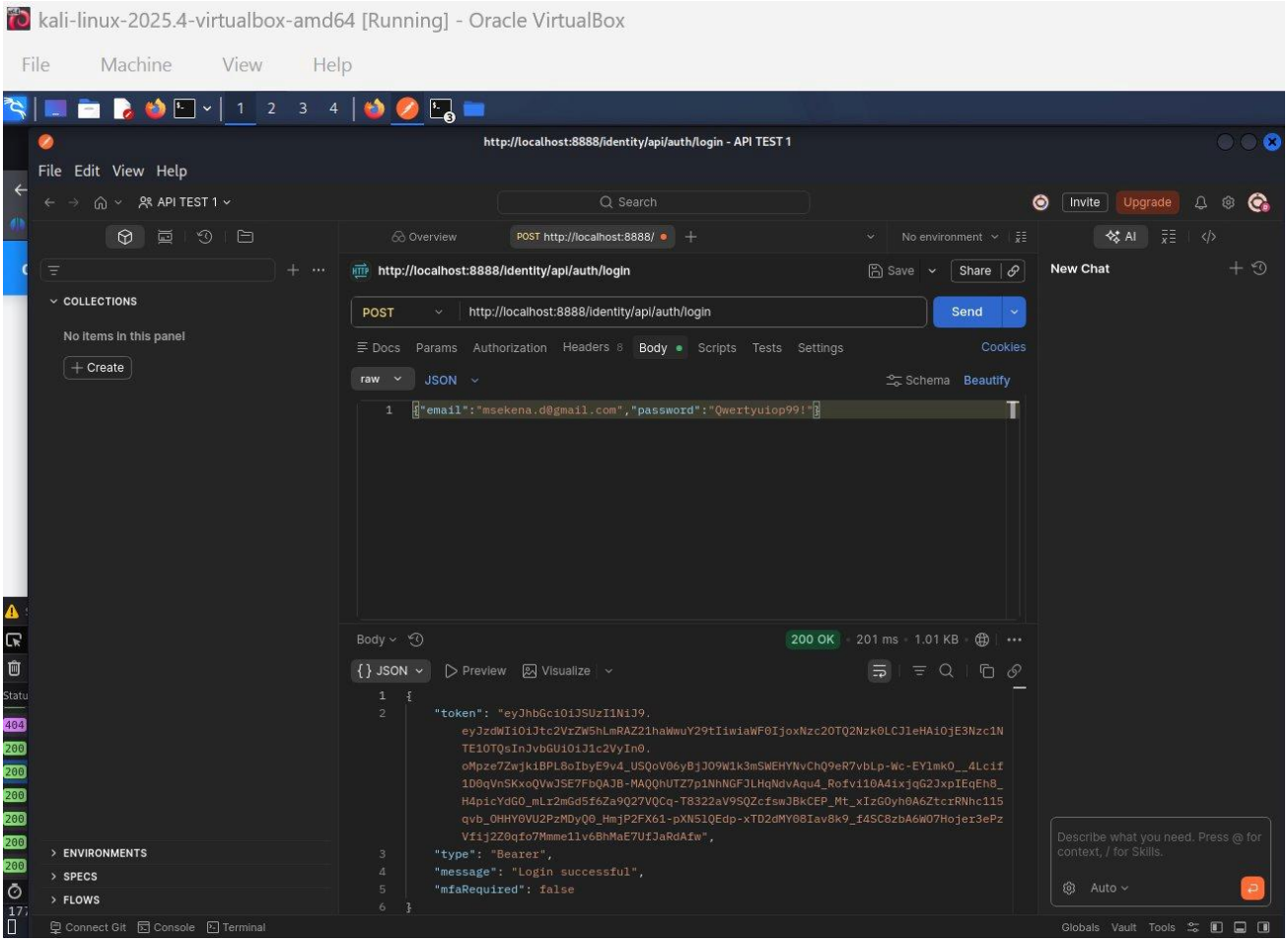

*Figure 1: crAPI Sign-In Page*

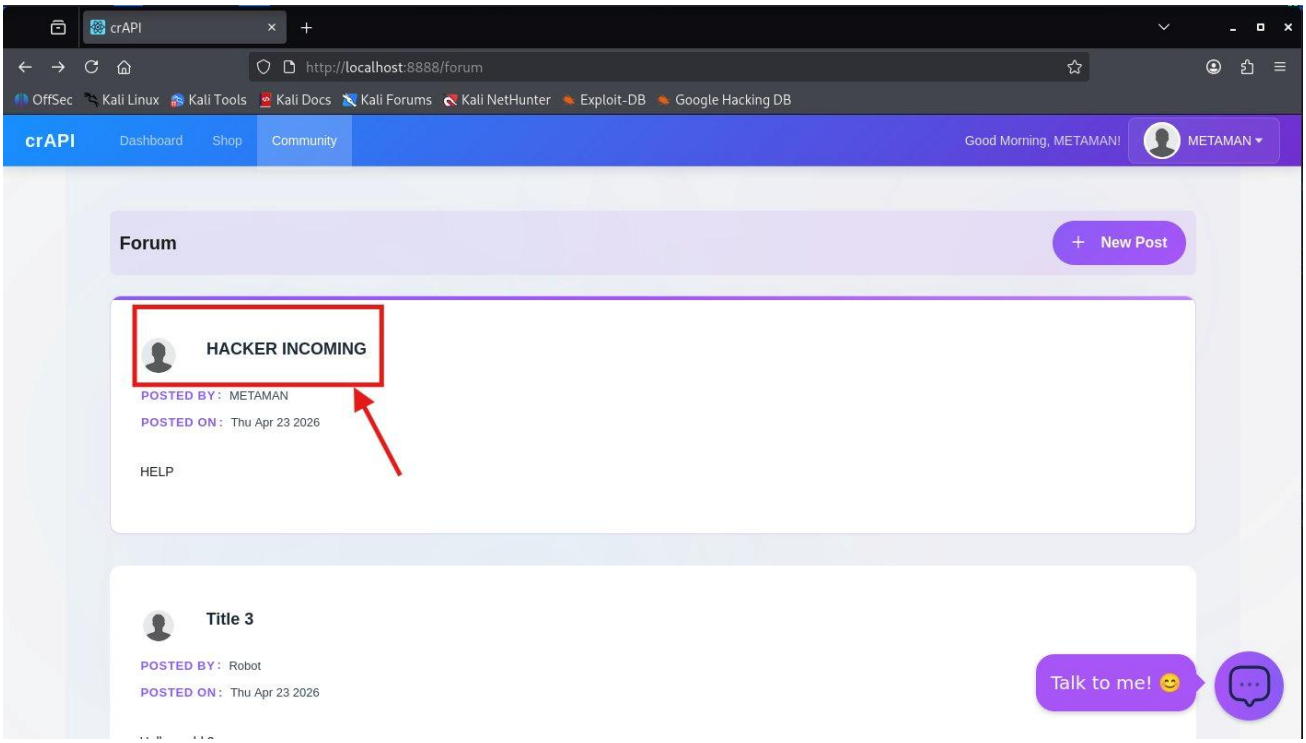


*Figure 2: crAPI Sign-Up / Community Page*

## 3. Scope

The scope of this security assessment was limited to the OWASP crAPI application running in a local Docker environment. The assessment was strictly black-box and grey-box in nature, simulating the perspective of an external attacker with limited prior knowledge of the system internals.

| | |
|---|---|
| **Target Application** | OWASP crAPI |
| **Environment** | Docker Container (localhost:8888) |
| **Assessment Type** | Manual API Security Assessment |
| **Methodology** | OWASP API Security Testing Guide |
| **In Scope** | All API endpoints exposed by crAPI including identity, community, vehicle, and shop APIs |
| **Out of Scope** | Network infrastructure, operating system, and third-party dependencies |
| **Tools Used** | Postman, Browser Developer Tools (Chrome/Firefox) |
| **Testing Period** | April 2026 |

## 4. Methodology

The assessment followed a structured manual testing approach guided by the OWASP API Security Testing Guide. The process was divided into the following phases:

**Reconnaissance & Endpoint Discovery**

Identifying all exposed API endpoints by observing network traffic, analyzing API responses.

**Authentication & Authorization Testing**

Testing for broken authentication mechanisms, weak token generation, missing access controls, and privilege escalation vectors.

**Input Validation Testing**

Submitting unexpected and unauthorized parameters to identify mass assignment, injection, or improper input handling vulnerabilities.

**Rate Limiting & Resource Abuse**

Sending repeated requests to identify missing rate limits and brute force protections.

**Information Disclosure Analysis**

Reviewing API responses for sensitive data leakage including internal paths, tokens, and system metadata.

**Documentation & Reporting**

Recording all findings with screenshots, OWASP classification, severity scoring, and remediation guidance.

## 5. Vulnerabilities Found

The following sections detail each vulnerability identified during the assessment. Each finding includes the affected endpoint, a description of the vulnerability, its potential impact, supporting evidence, and recommended remediation steps.

### 5.1. API Endpoint Discovery

**Severity:**

**MEDIUM**

During the reconnaissance phase, I was able to enumerate multiple internal API endpoints by passively observing browser network requests. The crAPI application makes API calls to backend endpoints that are fully visible in the browser's Developer Tools Network tab, including endpoints for product listings and order history.

| | |
|---|---|
| **Vulnerability** | API Endpoint Discovery |
| **Endpoint** | GET /workshop/api/shop/products \| GET /workshop/api/shop/orders |
| **Impact** | Exposing internal API endpoints allows attackers to map the entire attack surface of the application, facilitating further exploitation of other vulnerabilities such as BOLA, injection attacks, and data extraction. |
| **Evidence** | API endpoints were discovered by observing network traffic in the browser Developer Tools. Endpoints for Products and Orders were visible in plain text, revealing the full API structure without authentication prompts on some routes. |

**Remediation Steps:**

- Implement API gateway rate limiting and endpoint monitoring to detect automated enumeration attempts.
- Ensure all sensitive endpoints require proper authentication before returning data.
- Avoid exposing verbose API paths that reveal internal object names or data structures.

**Evidence Screenshots:**

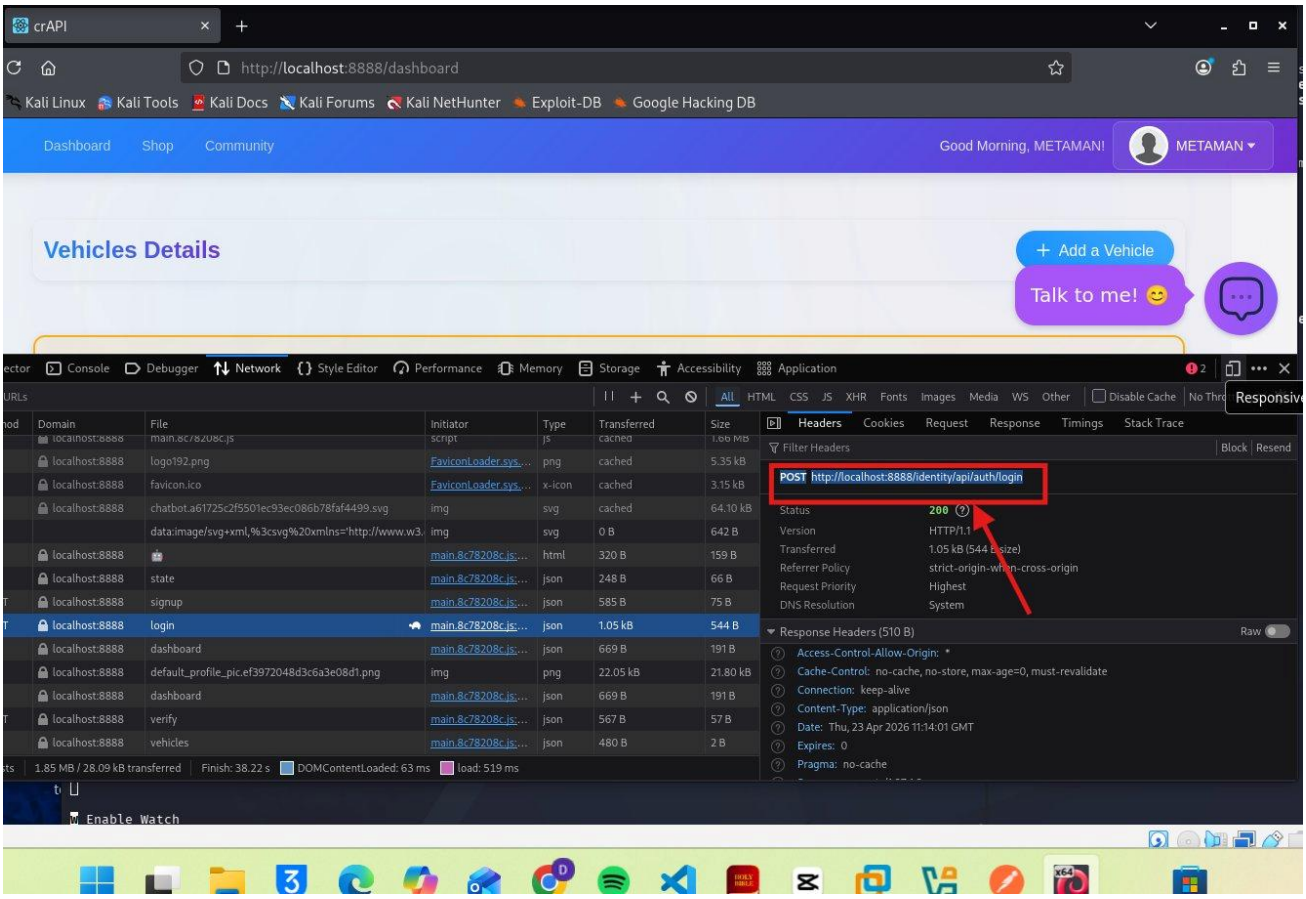


*Figure: API Endpoint Discovery — Evidence 1*

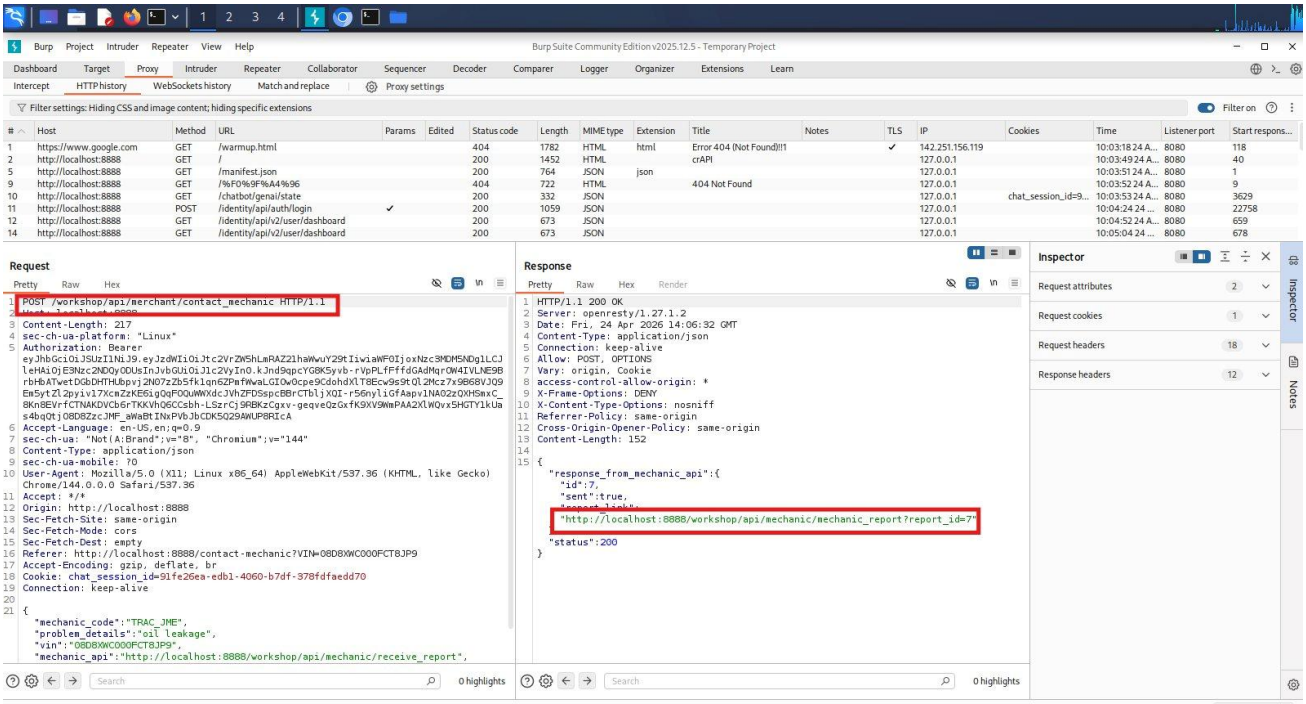


*Figure: API Endpoint Discovery  Evidence 2*

### 5.2. Broken Object Level Authorization (BOLA)

**Severity:**

**HIGH**

Broken Object Level Authorization (BOLA), also known as Insecure Direct Object Reference (IDOR), is the most prevalent and dangerous API vulnerability as ranked by OWASP. The API fails to validate whether the authenticated user is authorized to access the specific object being requested. By simply changing the vehicleId parameter in the request URL, I was able to retrieve vehicle details belonging to a completely different user account.

| | |
|---|---|
| **Vulnerability** | Broken Object Level Authorization (BOLA) |
| **Endpoint** | GET /workshop/api/vehicles/{vehicleId}/location |
| **Impact** | An authenticated attacker can access vehicle data belonging to other users by substituting their own vehicleId with that of another user. This constitutes a direct violation of horizontal access control and can expose sensitive location and vehicle information. |
| **Evidence** | I retrieved their vehicleId from the application, then observed another user's vehicleId through the browser Network tab. By substituting the foreign vehicleId in a Postman request, the API returned full vehicle details for the other user responding with 200 OK. |

**Remediation Steps:**

- Implement server-side object-level authorization checks which verify that the requesting user owns the resource being accessed before returning data.
- Log and alert on access attempts where a user requests objects not associated with their account.

**Evidence Screenshots:**

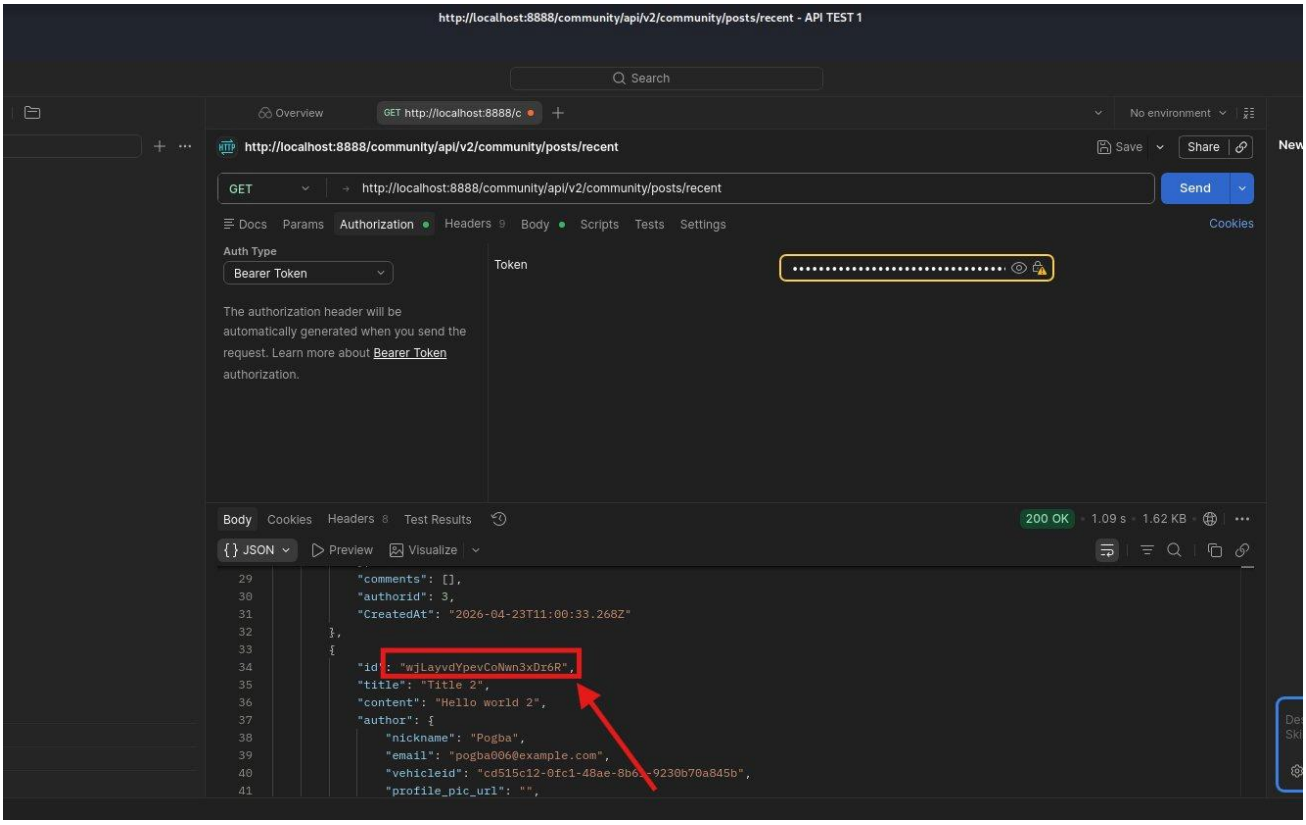


*Figure: Broken Object Level Authorization (BOLA) Evidence 1*

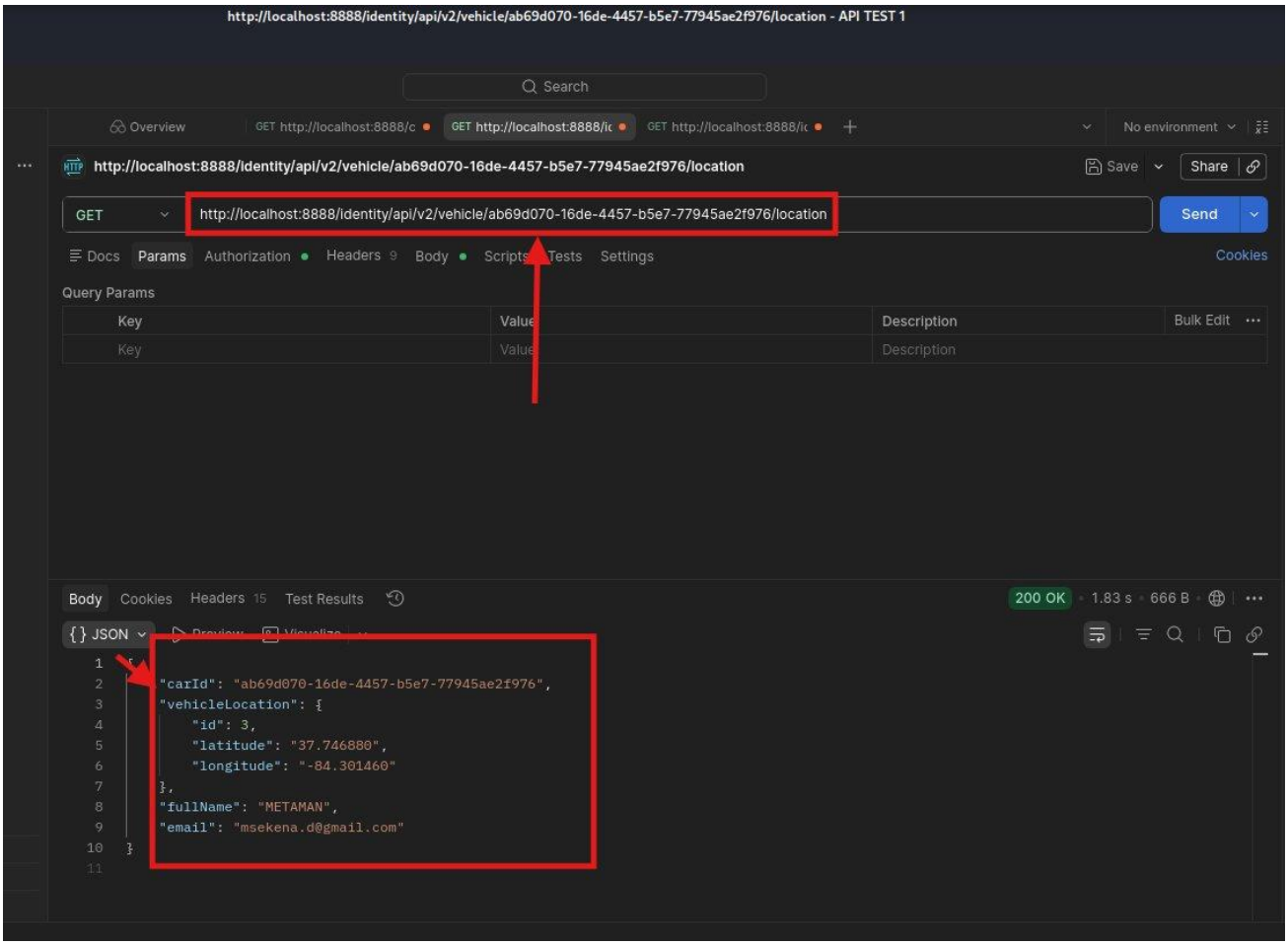


*Figure: Broken Object Level Authorization (BOLA) — Obtaining Another User Vehicle ID Evidence 2*

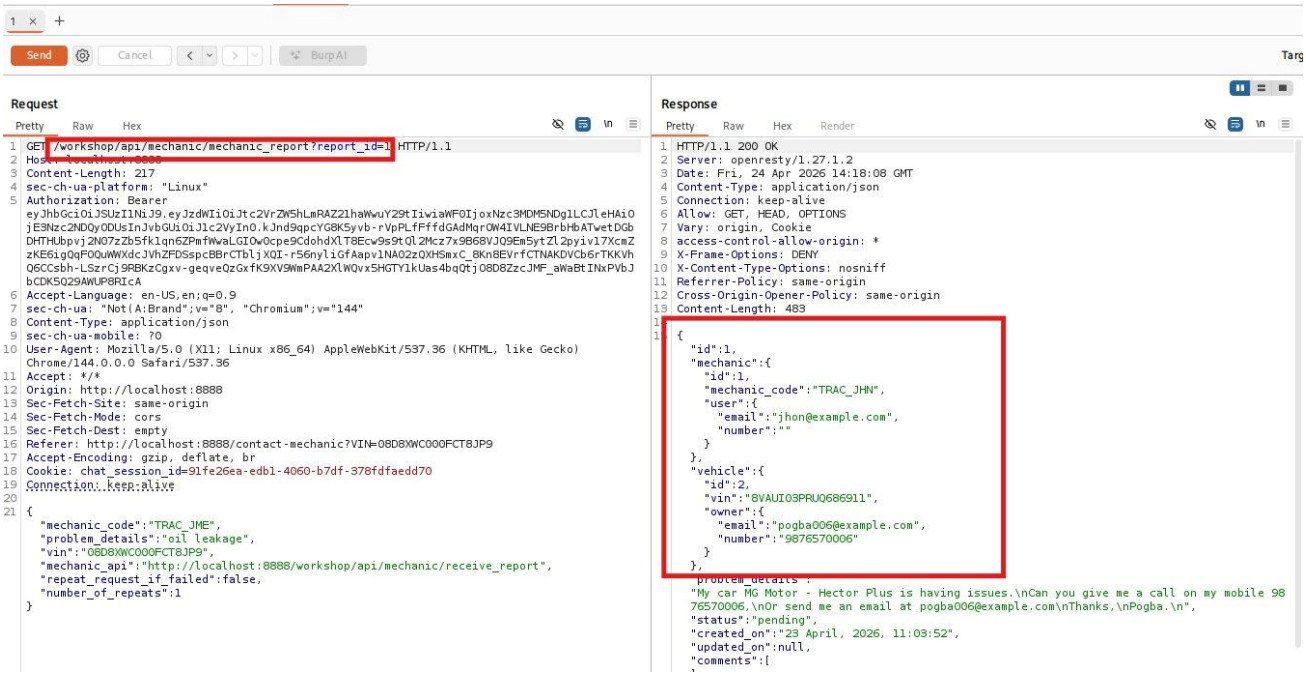

*Figure: Broken Object Level Authorization (BOLA) Evidence 3*

### 5.3. Broken Authorization (Accessing Other Users' Data)

**Severity:**

**HIGH**

This vulnerability specifically demonstrates that the authorization failure extends to general car data retrieval endpoints. The lack of ownership enforcement means any authenticated user can retrieve any other user's vehicle information by simply knowing or guessing their car ID.

| | |
|---|---|
| **Vulnerability** | Broken Authorization, Accessing Other Users' Data |
| **Endpoint** | GET /workshop/api/vehicles/{carId} |
| **Impact** | Attackers can access personal data of any user in the system simply by iterating over known or guessable car IDs. This leads to mass data harvesting and potential identity theft if sensitive PII is returned in the response. |
| **Evidence** | By changing the Car ID value in the API request, I was able to retrieve vehicle details associated with a different user account. The server returned a 200 OK response with the target user's vehicle data, confirming the absence of ownership verification. |

**Remediation Steps:**

- Enforce strict access control checks at the API layer where every data access request must validate that the authenticated session owner is the resource owner.
- Avoid using sequential or predictable identifiers for objects that contain sensitive information.
- Return 403 Forbidden (not 404) when a user attempts to access a resource they do not own, to prevent enumeration.

**Evidence Screenshots:**

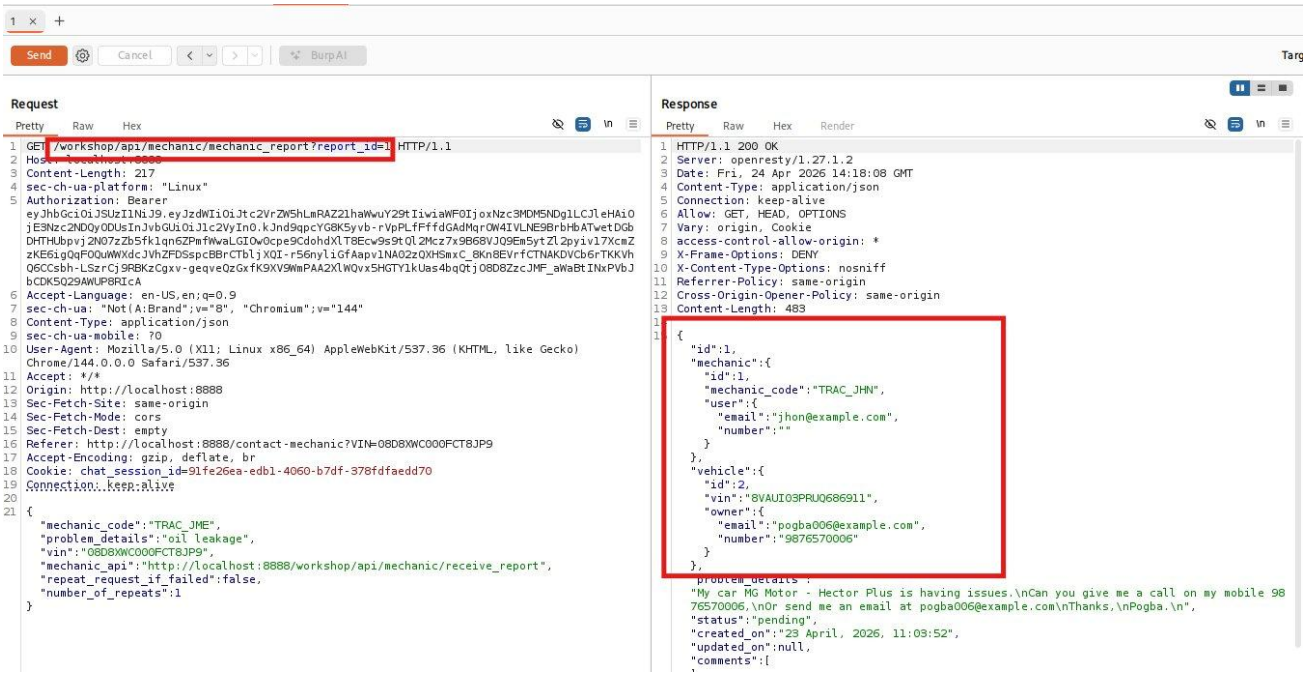

*Figure: Broken Authorization — Accessing Other Users' Data — Evidence 1*

### 5.4. Excessive Data Exposure / Information Disclosure

**Severity:**

**HIGH**

Excessive Data Exposure occurs when an API endpoint returns more information than necessary, relying on the client application to filter what is displayed. In crAPI, the community posts endpoint reveals user data that is not rendered in the frontend interface but is fully accessible in the raw API response.

| | |
|---|---|
| **Vulnerability** | Excessive Data Exposure / Information Disclosure |
| **Endpoint** | GET /community/api/v2/community/posts/recent?limit=30&offset=0 |
| **Impact** | The API returns more data than is necessary for the client's intended function. Sensitive fields such as user identifiers, email addresses, vehicle IDs, and internal system metadata may be included in responses, enabling attackers to collect PII and escalate other attacks. |
| **Evidence** | A GET request to the community posts endpoint returned a verbose JSON response containing user data beyond what is displayed on the frontend. Fields visible in the raw API response were not shown in the UI, indicating that filtering is done client-side rather than at the API/server level. |

**Remediation Steps:**

- Apply the principle of data minimization — return only the fields that the client application actively needs.
- Implement server-side response filtering rather than relying on the frontend to hide sensitive fields.
- Regularly audit API responses for unintended data exposure using automated tools.

**Evidence Screenshots:**

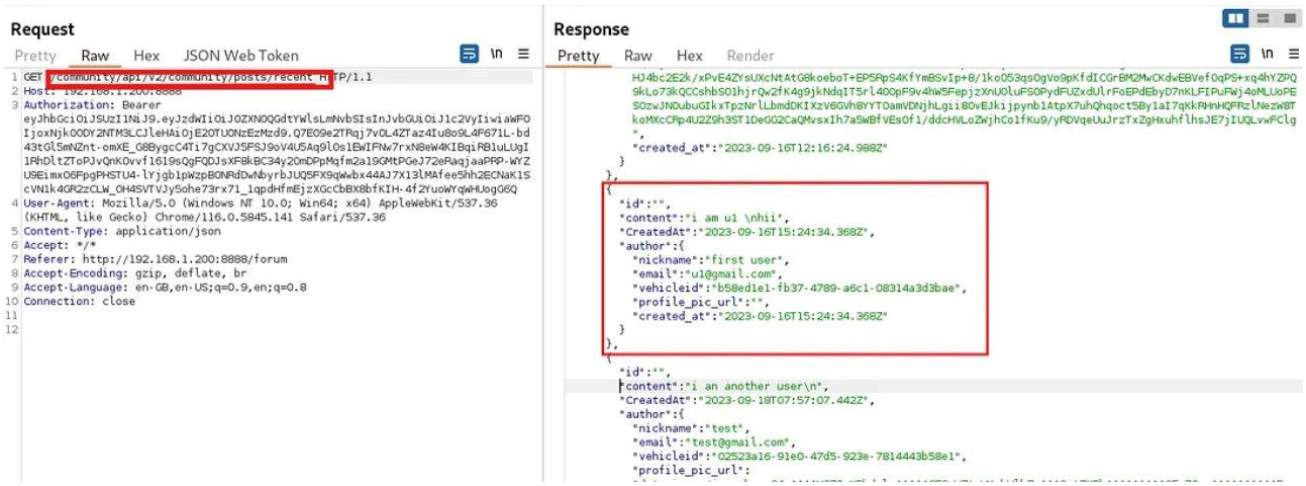


*Figure: Excessive Data Exposure / Information Disclosure — Evidence 1*

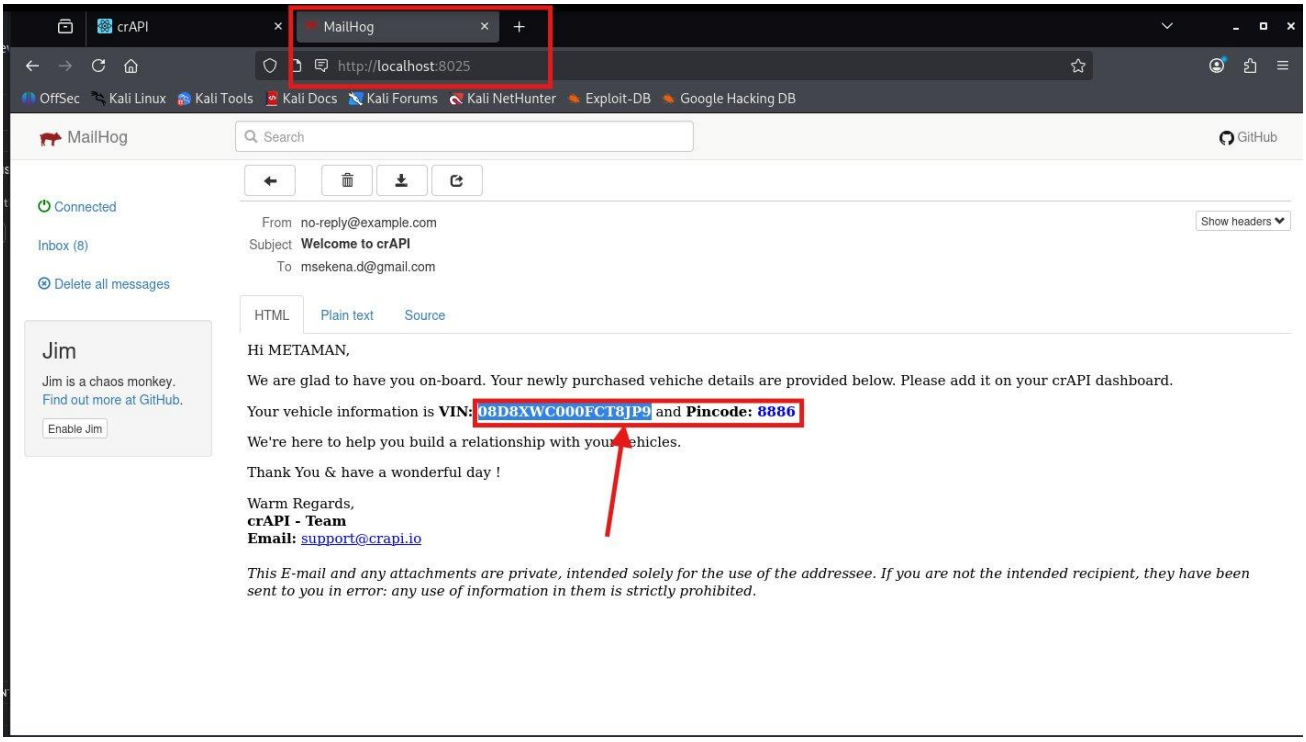


*Figure: Excessive Data Exposure — MailHog Sensitive Data Disclosure — Evidence 2*

### 5.5. Weak Tokens (JWT & OTP Brute Force)

**Severity:**

**CRITICAL**

Broken Authentication is a vulnerability that allows attackers to compromise authentication tokens or mechanisms. Weak JWT secrets can be cracked offline using tools such as hashcat or jwtcracker, while unprotected OTP endpoints are trivially brute-forced. Either pathway leads to full account takeover without requiring knowledge of the victim's password.

| | |
|---|---|
| **Vulnerability** | Weak Tokens (JWT & OTP Brute Force) |
| **Endpoint** | POST /identity/api/auth/login |
| **Impact** | Weak or predictable tokens enable brute force attacks against authentication mechanisms. An attacker can bypass login controls by repeatedly guessing OTP values or exploiting weak JWT signatures, leading to full account takeover. |
| **Evidence** | The login endpoint was observed to accept repeated authentication attempts without triggering any lockout or challenge mechanism. The JWT token structure was analyzed and found to use a weak signing algorithm susceptible to offline cracking. The OTP mechanism does not implement attempt limits, making it vulnerable to brute force enumeration. |

**Remediation Steps:**

- Enforce account lockout or progressive delays after a configurable number of failed authentication attempts (e.g., 5 attempts).
- Use strong JWT signing algorithms (RS256 or ES256) and validate token integrity on every request.
- Use multi-factor authentication (MFA) for sensitive operations beyond initial login.

**Evidence Screenshots:**

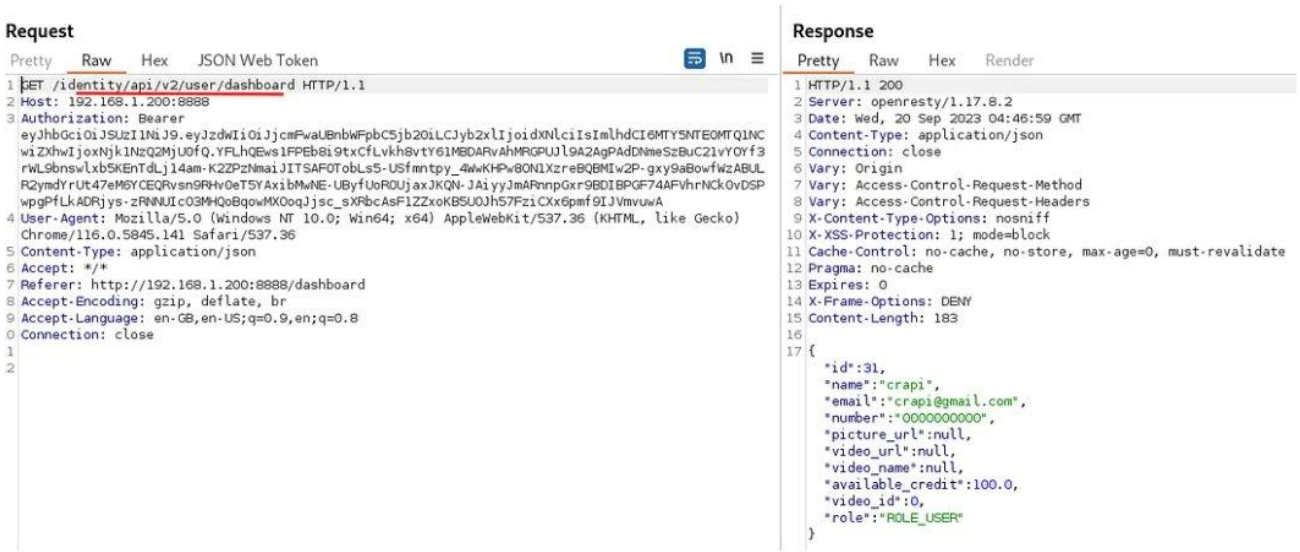


*Figure: Weak Tokens — JWT & OTP Brute Force — Evidence 1*

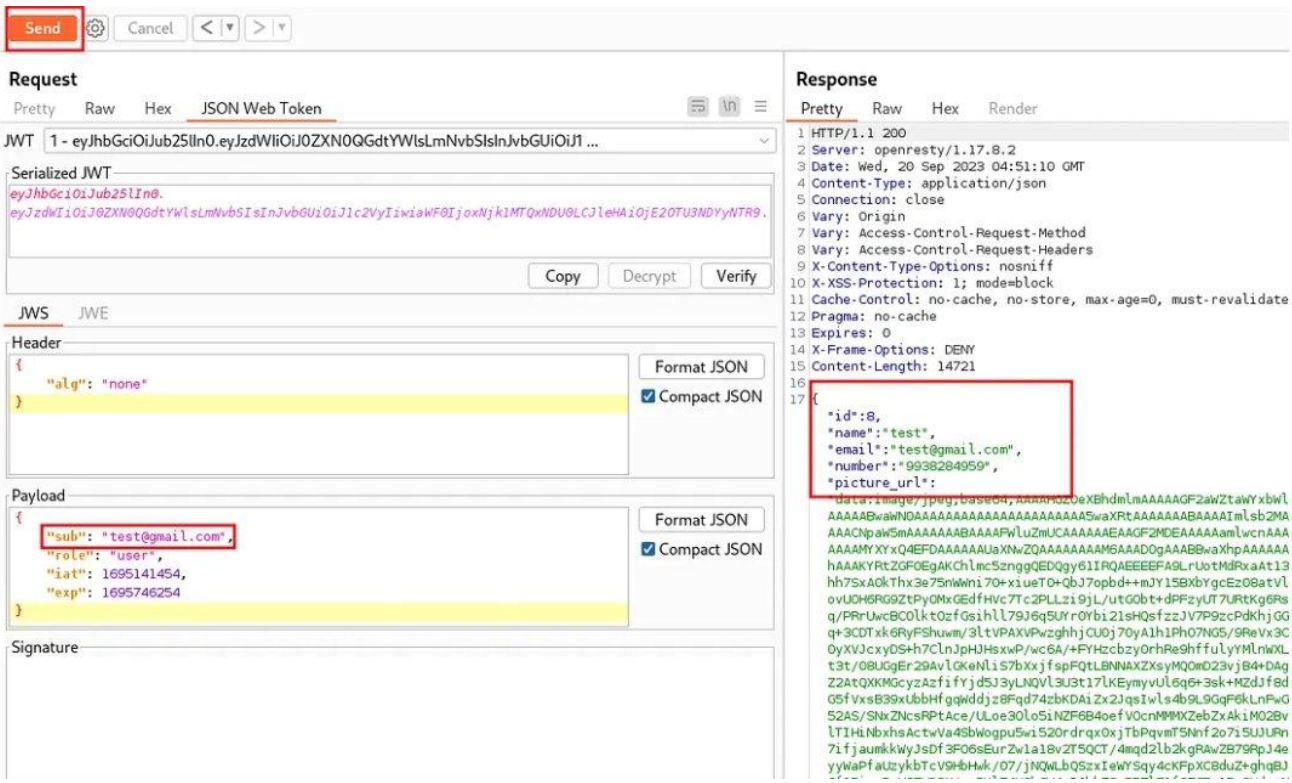


*Figure: Weak Tokens — JWT Forged Token (alg: none) — Evidence 2*

### 5.6. Missing Rate Limiting

**Severity:**

**HIGH**

Missing Rate Limiting is a vulnerability that leaves APIs open to abuse through automated scripting. The crAPI application imposes no restrictions on the frequency of API requests, meaning an attacker can enumerate all community posts, perform credential stuffing attacks, or overload the server without any resistance. The repeated 200 OK responses with consistent response times confirm this finding.

| | |
|---|---|
| **Vulnerability** | Missing Rate Limiting |
| **Endpoint** | GET /community/api/v2/community/posts/recent?limit=30&offset=0 |
| **Impact** | Without rate limiting, attackers can send an unlimited number of requests to enumerate data, brute force credentials, scrape sensitive information, or cause a Denial of Service (DoS) condition by exhausting server resources. |
| **Evidence** | The community posts endpoint was queried repeatedly in rapid succession. In all cases, the server responded with 200 OK without any slowdown or blocking. No rate limit headers (X-RateLimit-Limit, Retry-After) were present in the HTTP response, confirming the absence of request throttling. |

**Remediation Steps:**

- Implement server-side rate limiting using an API gateway or middleware (e.g., Nginx rate limiting, AWS API Gateway throttling).
- Add rate limit response headers (X-RateLimit-Limit, X-RateLimit-Remaining) to all API responses.

**Evidence Screenshots:**

**Request**

Pretty Raw Hex JSON Web Token

```
1 POST /workshop/api/merchant/contact_mechanic HTTP/1.1
2 Host: 192.168.1.200:8888
3 Content-Length: 214
4 Authorization: Bearer
  eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1M0BnbWFpbC5jb20iLCJyb2xlIjoidXNlciIsImlhdCI6
  MTY5NDg0MzM2MywiZXhwIjoxNjk1NDQ4MTYzfQ.CyVPtf8ZHKajBK0dqR2kPyKoXhJIt4aN2NENIH
  olfseU5fs0ahxR-KEBaLSLhmwjBWYQ55B8-2PBPJGDiuYKr6rK9plnosBU-UL3LbMgoAihIjvIJTV
  HS1Fn6lxnBBiW3lrwH1Zd7fUWEZgYLkKlpCYrn3Wjd0XiIu6CZ0l9PTtuUMzEwmsZz-elF-VCxK5b
  pT5fs_fPILAslwRemikrKnUdRdXCaTjpezxbotFh5rnpLfEBbUnspXxTsncw4GaqoZraVk3WwfLvy
  UU-kEjfRSLg6pf8s69EvdHiTabTYrEYpK5YNYE0cbcKJbF_YxJ--_sL1tovX3rPrCkGT3gIKA
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/116.0.5845.141 Safari/537.36
6 Content-Type: application/json
7 Accept: */*
8 Origin: http://192.168.1.200:8888
9 Referer: http://192.168.1.200:8888/contact-mechanic?VIN=0YC0M86AXXM220875
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
12 Connection: close
13
14 {
    "mechanic_code":"TRAC_JHN",
    "problem_details":"demo",
    "vin":"9RHHJ30LIPL081759",
    "mechanic_api":
    "http://192.168.1.200:8888/workshop/api/mechanic/receive_report",
    "repeat_request_if_failed":false,
    "number_of_repeats":1
  }
```

**Response**

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: openresty/1.17.8.2
3 Date: Tue, 19 Sep 2023 11:07:41 GMT
4 Content-Type: application/json
5 Connection: close
6 Allow: POST, OPTIONS
7 Vary: origin, Cookie
8 access-control-allow-origin: *
9 X-Frame-Options: DENY
10 X-Content-Type-Options: nosniff
11 Referrer-Policy: same-origin
12 Content-Length: 158
13
14 {
    "response_from_mechanic_api":{
      "id":22,
      "sent":true,
      "report_link":"http://192.168.1.200:8888/workshop/api/mechanic/mechanic_report?report_id=22"
    },
    "status":200
  }
```

*Figure: Missing Rate Limiting — Evidence 1*

**Request**

Pretty Raw Hex JSON Web Token

```
1 POST /workshop/api/merchant/contact_mechanic HTTP/1.1
2 Host: 192.168.1.200:8888
3 Content-Length: 217
4 Authorization: Bearer
  eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1M0BnbWFpbC5jb20iLCJyb2xlIjoidXNlciIsImlhdCI6
  MTY5NDg0MzM2MywiZXhwIjoxNjk1NDQ4MTYzfQ.CyVPtf8ZHKajBK0dqR2kPyKoXhJIt4aN2NENIH
  olfseU5fs0ahxR-KEBaLSLhmwjBWYQ55B8-2PBPJGDiuYKr6rK9plnosBU-UL3LbMgoAihIjvIJTV
  HS1Fn6lxnBBiW3lrwH1Zd7fUWEZgYLkKlpCYrn3Wjd0XiIu6CZ0l9PTtuUMzEwmsZz-elF-VCxK5b
  pT5fs_fPILAslwRemikrKnUdRdXCaTjpezxbotFh5rnpLfEBbUnspXxTsncw4GaqoZraVk3WwfLvy
  UU-kEjfRSLg6pf8s69EvdHiTabTYrEYpK5YNYE0cbcKJbF_YxJ--_sL1tovX3rPrCkGT3gIKA
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/116.0.5845.141 Safari/537.36
6 Content-Type: application/json
7 Accept: */*
8 Origin: http://192.168.1.200:8888
9 Referer: http://192.168.1.200:8888/contact-mechanic?VIN=0YC0M86AXXM220875
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
12 Connection: close
13
14 {
    "mechanic_code":"TRAC_JHN",
    "problem_details":"demo",
    "vin":"9RHHJ30LIPL081759",
    "mechanic_api":
    "http://192.168.1.200:8888/workshop/api/mechanic/receive_report",
    "repeat_request_if_failed":true,
    "number_of_repeats":10000
  }
```

**Response**

Pretty Raw Hex

```
1 HTTP/1.1 503 Service Unavailable
2 Server: openresty/1.17.8.2
3 Date: Tue, 19 Sep 2023 13:25:33 GMT
4 Content-Type: application/json
5 Connection: close
6 Allow: POST, OPTIONS
7 Vary: origin, Cookie
8 access-control-allow-origin: *
9 X-Frame-Options: DENY
10 X-Content-Type-Options: nosniff
11 Referrer-Policy: same-origin
12 Content-Length: 71
13
14 {
    "message": "Service unavailable. Seems like you caused layer 7 DoS :)"
  }
```

*Figure: Missing Rate Limiting / DoS Condition — Evidence 2*

### 5.7. Mass Assignment

**Severity:**

**CRITICAL**

Mass Assignment (Unrestricted Access) occurs when an API automatically binds client-supplied parameters to internal object properties without filtering. In this case, the crAPI signup endpoint accepts the entire JSON request body and maps it directly to the user object, including sensitive fields such as role and isAdmin. This is a critical vulnerability as it allows any user to self-elevate their privileges at registration time.

| | |
|---|---|
| **Vulnerability** | Mass Assignment / Input Validation Failure |
| **Endpoint** | POST /identity/api/auth/signup |
| **Impact** | An attacker can self-assign privileged roles (e.g., ADMIN) and account types (e.g., premium) during account registration, bypassing intended access controls. This leads to unauthorized privilege escalation, giving the attacker administrative access to the application and its data. |
| **Evidence** | A signup request was crafted with additional fields not expected by the standard registration form: role: ADMIN, isAdmin: true, and accountType: premium. The server accepted these fields and responded with 200 OK 'User registered successfully'. This confirms that the server binds all user-supplied JSON fields directly to the underlying data model without filtering or validation. |

**Remediation Steps:**

- Implement a strict allowlist (DTO — Data Transfer Object pattern) on the server to only accept explicitly defined fields: name, number, email, password.
- Reject requests containing unexpected or unauthorized fields with a 400 Bad Request response.
- Conduct thorough input validation on all API endpoints, rejecting unknown parameters.

**Evidence Screenshots:**

**Request**

Pretty Raw Hex JSON Web Token

```
1 PUT /workshop/api/shop/orders/52 HTTP/1.1
2 Host: 192.168.1.200:8888
3 Authorization: Bearer
eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJzaG9wQGV4YW1wbGUuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2OTUyNzM2ND
AsImV4cCI6MTY5NTg3ODQ0MH0.Tsi9QyRB2wWC4_mVvs2LoBzOjsMVQAKxpVJ8QixvYX-EWji2zEEUweSGtwGaxt_XyE1Kn
FuOXBAbFBXJfxO6Mt3rQuE2HYfm9_TtMxgVEdVpqDrZXBaxOxd-wSmP4fJfCX6_oHpghbkFlMcJhddcb9HQuaCPx4spSkMy
9EcYcRO4k_gh8k8r4v7cNEWhVjCwH7SiP4PXGCbdsfOiHRzacfOjCAaT34qEExKIyLspeXTmgNEkHJUZHnxeynoCVORiVvp
II6BTz4pj-XrBI1QwS9zaFkCRF4sIQ-inmdk35AI_bf3WROljKWB9trUdDnXBNgPUFelax1cNilhSACmtEw
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/116.0.5845.141 Safari/537.36
5 Content-Type: application/json
6 Accept: */*
7 Referer: http://192.168.1.200:8888/orders?order_id=52
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
10 Connection: close
11 Content-Length: 19
12
13 {
  "status":"return"
}
```

**Response**

Pretty Raw Hex Render

```
1 HTTP/1.1 400 Bad Request
2 Server: openresty/1.17.8.2
3 Date: Thu, 21 Sep 2023 05:34:53 GMT
4 Content-Type: application/json
5 Connection: close
6 Allow: GET, POST, PUT, HEAD, OPTIONS
7 Vary: origin, Cookie
8 X-Frame-Options: DENY
9 X-Content-Type-Options: nosniff
10 Referrer-Policy: same-origin
11 Content-Length: 88
12
13 {
  "message":"The value of 'status' has to be 'delivered','return pending' or 'returned'"
}
```

*Figure: Mass Assignment / Input Validation Failure — Evidence 1*

**Request**

Pretty Raw Hex JSON Web Token

```
1 PUT /workshop/api/shop/orders/53 HTTP/1.1
2 Host: 192.168.1.200:8888
3 Authorization: Bearer
eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJzaG9wQGV4YW1wbGUuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2OTUyNzM2ND
AsImV4cCI6MTY5NTg3ODQ0MH0.Tsi9QyRB2wWC4_mVvs2LoBzOjsMVQAKxpVJ8QixvYX-EWji2zEEUweSGtwGaxt_XyE1Kn
FuOXBAbFBXJfxO6Mt3rQuE2HYfm9_TtMxgVEdVpqDrZXBaxOxd-wSmP4fJfCX6_oHpghbkFlMcJhddcb9HQuaCPx4spSkMy
9EcYcRO4k_gh8k8r4v7cNEWhVjCwH7SiP4PXGCbdsfOiHRzacfOjCAaT34qEExKIyLspeXTmgNEkHJUZHnxeynoCVORiVvp
II6BTz4pj-XrBI1QwS9zaFkCRF4sIQ-inmdk35AI_bf3WROljKWB9trUdDnXBNgPUFelax1cNilhSACmtEw
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/116.0.5845.141 Safari/537.36
5 Content-Type: application/json
6 Accept: */*
7 Referer: http://192.168.1.200:8888/orders?order_id=52
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
10 Connection: close
11 Content-Length: 38
12
13 {
14   "quantity":100,
  "status":"returned"
}
```

**Response**

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: openresty/1.17.8.2
3 Date: Thu, 21 Sep 2023 05:39:40 GMT
4 Content-Type: application/json
5 Connection: close
6 Allow: GET, POST, PUT, HEAD, OPTIONS
7 Vary: origin, Cookie
8 X-Frame-Options: DENY
9 X-Content-Type-Options: nosniff
10 Referrer-Policy: same-origin
11 Content-Length: 290
12
13 {
  "orders":{
    "id":53,
    "user":{
      "email":"shop@example.com",
      "number":"0000000001"
    },
    "product":{
      "id":1,
      "name":"Seat",
      "price":"10.00",
      "image_url":"images/seat.svg"
    },
    "quantity":100,
    "status":"returned",
    "transaction_id":"6fa3e339-16ea-46c4-8c9f-a087f49698a5",
    "created_on":"2023-09-21T05:22:13.735232"
  }
}
```

*Figure: Mass Assignment — Alternate Method / Privilege Fields  Evidence 2*

**Request**

Pretty Raw Hex JSON Web Token

```
1 GET /workshop/api/shop/orders/52 HTTP/1.1
2 Host: 192.168.1.200:8888
3 Authorization: Bearer
  eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJzaG9wQGV4YW1wbGUuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2OTUyNzM2ND
  AsImV4cCI6MTY5NTg3ODQ0MHO.Tsi9QyRB2wWC4_mVvs2LoBzOjsMVQAKxpVJ8QixvYX-EWji2zEEUweSGtwGaxt_XyElKn
  FuOXBAbFBXJfx06Mt3rQuE2HYfm9_TtMxgVEdVpqDrZXBaxOxd-wSmP4fJfCX6_oHpghbkFlMcJhddcb9HQuaCPx4spSkMy
  9EcYcRO4k_gh8k8r4v7cNEWhVjCwH7SiP4PXGCbdsfOiHRzacfOjCAaT34qEExKIyLspeXTmgNEkHJUZHnxeynoCVORiVvp
  II6BTz4pj-XrBI1QwS9zaFkCRF4sIQ-inmdk35AI_bf3wROljKWB9trUdDnXBNgPUFelax1cNilhSACmtEw
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/116.0.5845.141 Safari/537.36
5 Content-Type: application/json
6 Accept: */*
7 Referer: http://192.168.1.200:8888/orders?order_id=52
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
10 Connection: close
11
12
```

**Response**

Pretty Raw Hex Render

```
4 Content-Type: application/json
5 Connection: close
6 Allow: GET, POST, PUT, HEAD, OPTIONS
7 Vary: origin, Cookie
8 X-Frame-Options: DENY
9 X-Content-Type-Options: nosniff
10 Referrer-Policy: same-origin
11 Content-Length: 545
12
13 {
    "order":{
      "id":52,
      "user":{
        "email":"shop@example.com",
        "number":"0000000001"
      },
      "product":{
        "id":1,
        "name":"Seat",
        "price":"10.00",
        "image_url":"images/seat.svg"
      },
      "quantity":1,
      "status":"delivered",
      "transaction_id":"3341c0ed-aa85-4002-84e4-01aee18698b4",
      "created_on":"2023-09-21T05:21:40.307555"
    },
    "payment":{
      "transaction_id":"3341c0ed-aa85-4002-84e4-01aee18698b4",
      "order_id":52,
      "amount":10,
      "paid_on":"2023-09-21T05:21:40.307555",
      "card_number":"XXXXXXXXXXXX5300",
      "card_owner_name":"shop",
      "card_type":"MasterCard",
      "card_expiry":"08/2027",
      "currency":"USD"
    }
  }
```

*Figure: Mass Assignment — Alternate Methods Revealed  Evidence 3*

## 6. Conclusion

This API security assessment of the OWASP crAPI platform identified seven vulnerabilities of varying severity levels, with two classified as Critical, four as High, and one as Medium. The findings collectively demonstrate a comprehensive range of real-world API security weaknesses, particularly in the areas of authorization, authentication, input handling, and resource management.

This assessment was conducted solely for academic purposes within a controlled environment as part of internship at Future Interns.

The OWASP API Security Top 10 framework provided a structured and industry-recognized foundation for classifying and communicating these vulnerabilities. The issues identified in crAPI directly mirror patterns observed in real production API environments, making this assessment a valuable exercise in developing practical API penetration testing skills.

All vulnerabilities documented in this report have known, implementable remediations. Addressing the Critical findings  particularly the JWT weakness and Mass Assignment — should be treated as a priority, followed by the High-severity authorization and data exposure issues.